Paper Review • arXiv 2610.40799

CaPtain

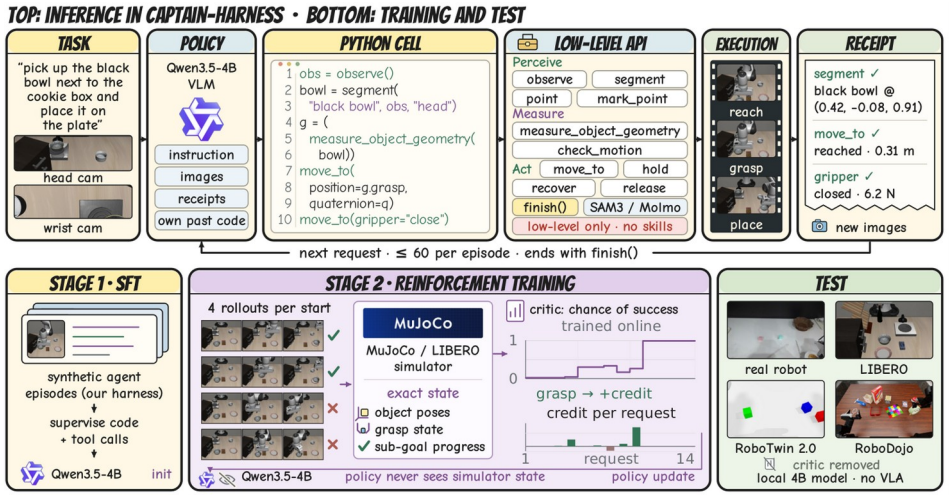
A Pipeline for Post-Training Code-as-Policy Robot Agents

Porting the language-agent SFT + multi-turn RL recipe to Code-as-Policy robots

A 4B open VLM coding with only low-level primitives

Yijia Fan, Xilin He, Yilei Jiang, Dongming Wu, Jiaxu Wang, Xiangyu Yue †
MMLab, The Chinese University of Hong Kong • NovaXBot

First submitted: 2026-10-29



Source: paper Figure 1 (method overview)

TL;DR: SFT+RL takes a 4B open VLM from 0% to a frontier-level CaP agent

With only 22 low-level primitives (no grasp/place skills, no VLA), **in-harness SFT followed by multi-turn RL with a critic reading privileged simulator physics** turns sparse episode outcomes into dense per-request credit; at test time: only camera images and receipts.

0% → 73.8%

LIBERO held-out success

Qwen3.5-4B 0-shot → SFT 55.4% → RL 73.8%

78.5%

LIBERO-PRO (3 suites)

Above K1 77.2 / WAA 75.6 / ASPIRE 71.7

+17.6~21.2

RL gains hold under perturbation

3 perturbed suites: LIBERO-PRO/Plus/MAX

83.3%

SimplerEnv WidowX zero-shot

No target data; π0.5 (Bridge FT) 57.1%

75.0

CaP-Bench zero-shot

Above Gemini-3-Pro 73.5, CaP-RL 7B 62.0

$0.03–0.06

Est. cost per episode

OpenRUA (Claude Opus 5): $8–22 / episode

Existing CaP agents avoid training: they rely on closed models or delegate manipulation to VLAs

① Prompt closed frontier models

- OpenRUA、 K1、 WAA、 ASPIRE …
- No weight updates; costly (OpenRUA $8–22/ep.)
- Can't fine-tune for new robots/tasks or run onboard

② Delegating to VLA/skills

- PyRUA、 AgenticRoboRL、 RoboRSI、 Gemini 4 Argon
- Code picks the skill; skill decides how to grasp/place
- Capped by skill data; RL learns sequencing at best

③ Small open VLMs fail outright

- Qwen3.5-4B: 0% LIBERO success in harness
- Prior training work mostly SFT (Guava)
- Or single-turn text RL (CaP-X, 3 Robosuite tasks)

Challenge: multi-turn credit assignment

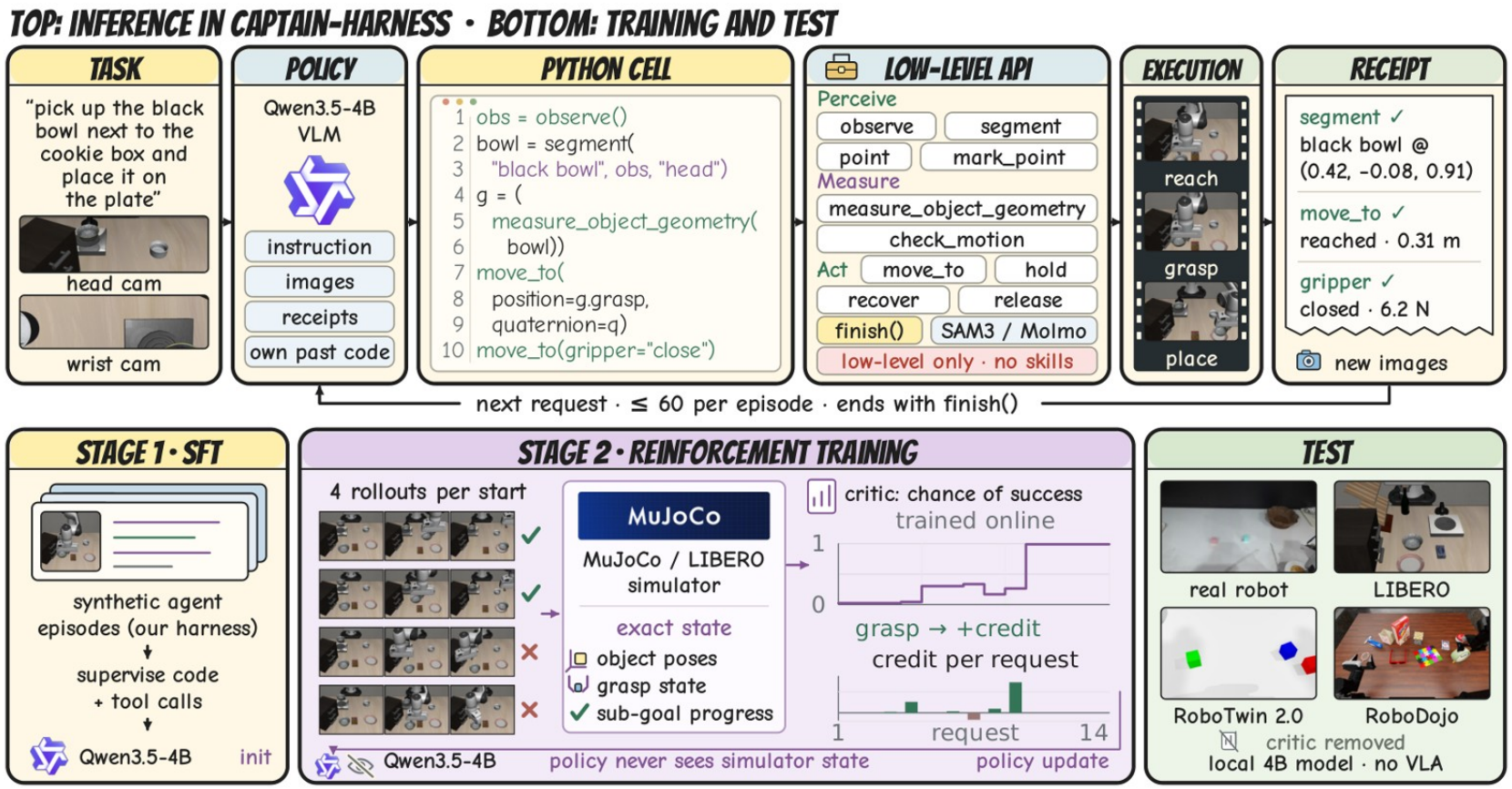
- No skills → policy picks grasp point, approach, gripper timing
- Dozens of requests/episode, mostly perception/checks
- Effects appear turns later; only one end-of-episode reward

Opportunity: asymmetric information in sim

- In sim, object poses, grasps, subgoal progress are exact
- At deployment the policy sees only cameras

→ Critic: privileged state; policy: images & receipts only

CaPtain overview: fixed harness + in-harness SFT + simulator-critic-guided multi-turn RL



Inference

Each request writes a Python cell → calls low-level APIs → reads receipt & new images; ⩽60 requests/episode, ends with finish()

Stage 1 • SFT

GPT-6 Astra synthetic trajectories in the same harness align the interaction format

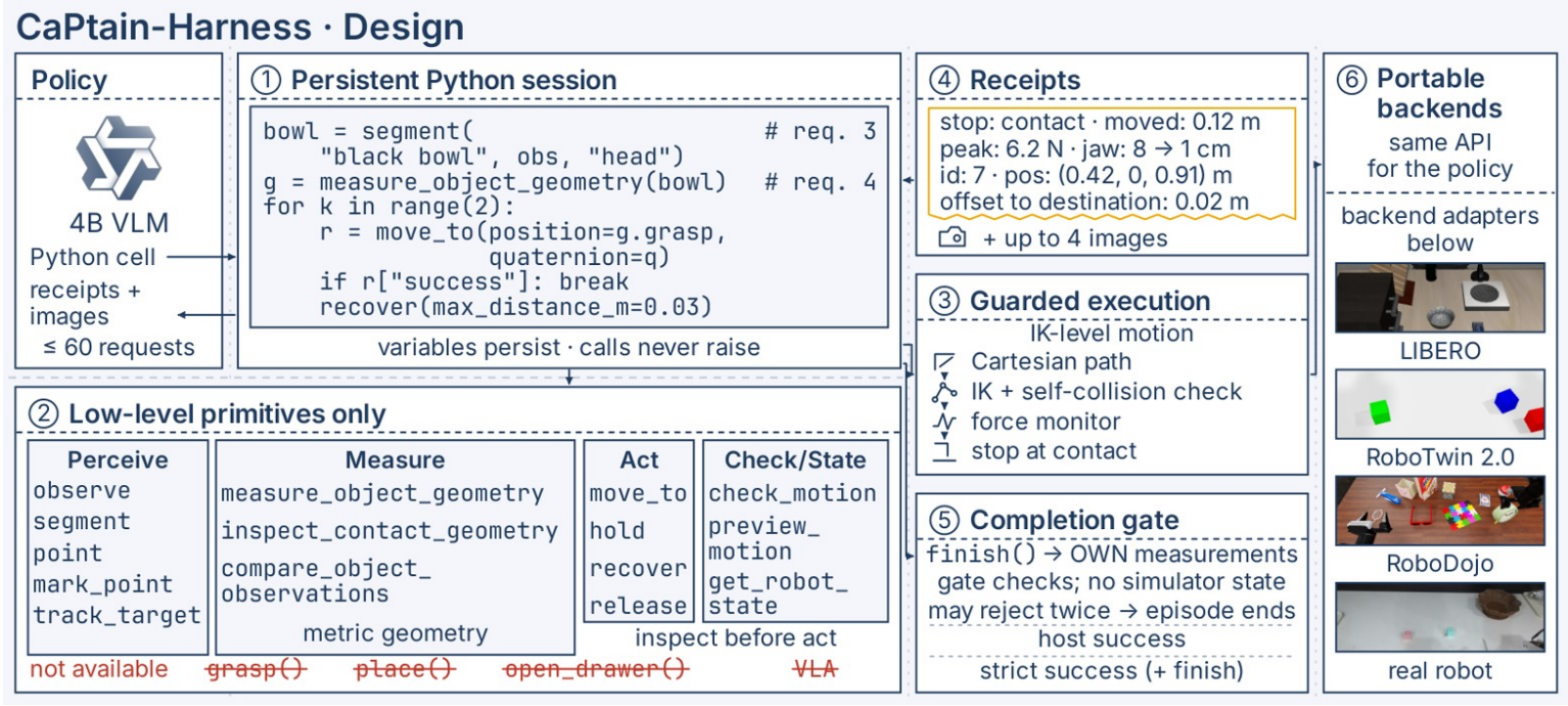
Stage 2 • RL

4 rollouts per start state; critic reads exact physics state and uses the change in success probability as per-request credit

Test

Critic removed; LIBERO / RoboTwin / RoboDojo / real robot; local 4B, no VLA

CaPtain-Harness: only low-level primitives; every manipulation decision lives in code



22 primitives + finish()

Perception / measurement / motion / planning & state; no grasp, place, open-drawer, nor VLA

Text receipts

⩽300 chars: stop reason, distance moved, peak force, gripper state, metric object coords & offset to target

Persistent Python session

Variables and self-written helpers persist across requests; calls never raise, so if / loops / retries work

Host vs. strict success

Strict = env-judged success + agent's own finish(); the deployable metric

Stage 1 SFT: frontier-agent trajectories teach the 4B model the interface (0% → 55.4%)

Format alignment

Chat template, TASK.md, tool schema and receipts identical to deployment; only assistant tokens supervised (reasoning + code)

Code normalization

226 “preview then move” variants in raw trajectories → unified into 4 helpers (val / need / downq / go), verified point-by-point on a mock robot

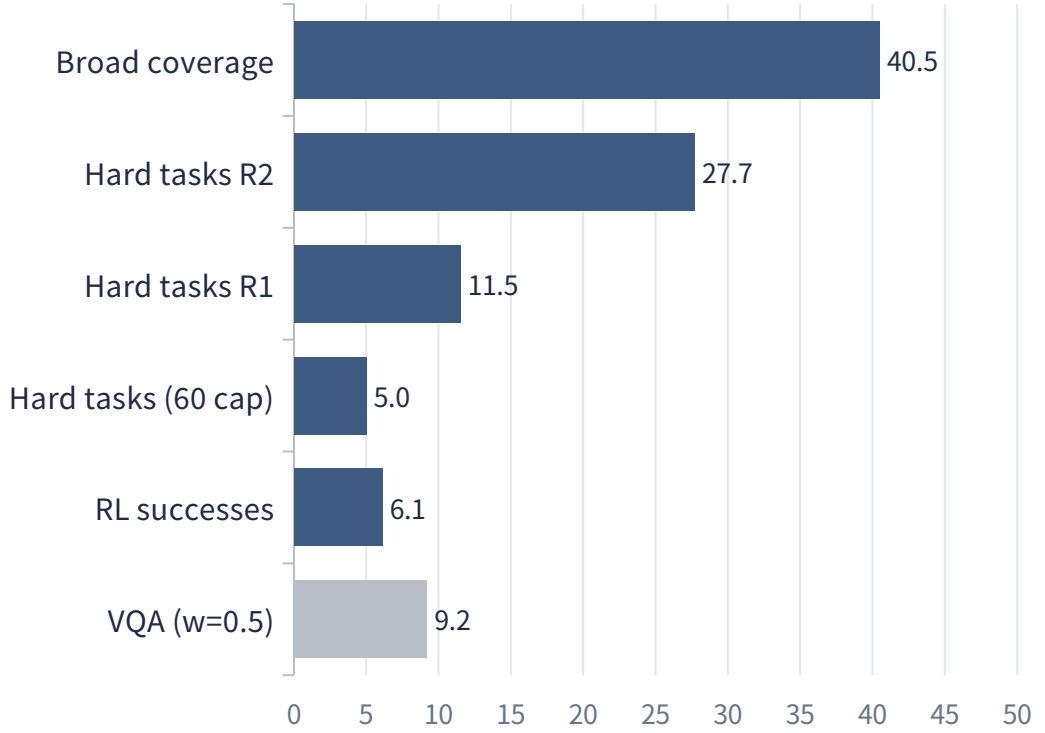
Forcing the model to look

mark_point pixel coords in history replaced by <masked> with p=0.2 to prevent copying old coordinates

Layout diversity

Official initial object-position std≈1.4 cm → model memorizes coords; only wide layouts (std≈15 cm) make it localize from images

SFT data mix (% of tokens)



25,961 rows • 861M tokens, 654M from GPT-6 Astra; full-parameter LLM tuning, frozen vision tower, 1.5 epochs

Stage 2: a critic reads privileged physics state and scores each request by its change in success probability

Privileged physics state $x(s_t)$

36-D: EE/gripper, goal progress, distances, object lift/displacement, joints & held flags
(excludes the policy's own action counts)

Critic V(s, t)

Ensemble of 5 MLPs predicts host success; warm-started on 1,000 old-policy episodes, then co-trained online with BCE

Per-request TD credit

$$\delta_t = c_t + V(s_{t+1}, \mathbf{t}) - V(s_t, \mathbf{t})$$

Both V use the same time index t → elapsed time earns no credit; pure perception $\delta \approx 0$

Advantage

Group outcome term (G=4 rollouts / start) + within-episode GAE (λ=0.9); shared by all tokens of a request

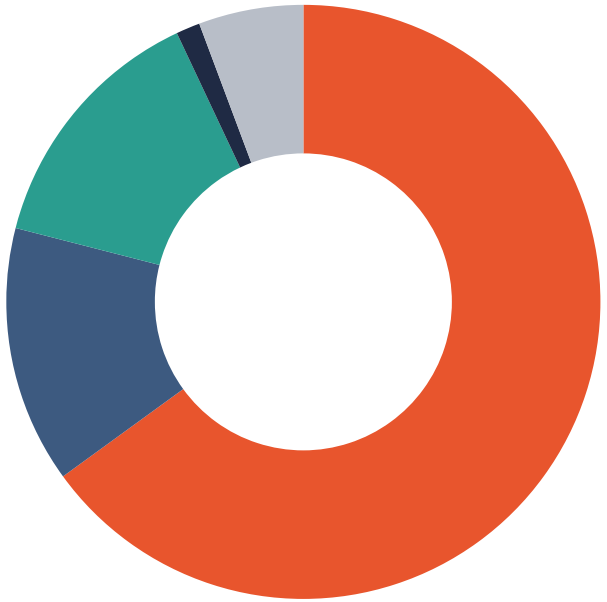
Ordering of terminal return R_i

Success + finish()	1.0
Success, no finish()	0.25
Partial subgoals (8 hard tasks)	$\leqslant 0.1125$
Other failures	0
Budget exhausted	−0.1
False finish	−0.3

- Step cost $c_t \leqslant 0$: failed or excess actions, post-success ops
- SAO-style streaming single-pass update; 2-sided token mask (0.2<r<4)
- 1,024-episode gradient accumulation per step; KL to SFT (β=0.002)

Credit concentrates on physical actions: perception requests get only 1.3% of advantage mass

|A| mass by request type



Motion 65% • Grasp 14% • Release 14% • **Pure perception 1.3%**

+0.058

Med. ΔV, good grasp

IQR [0.036, 0.090]

−0.0075

ΔV, failed grasp

−0.0002

ΔV, pure perception

≈0, regardless of token count

0.970

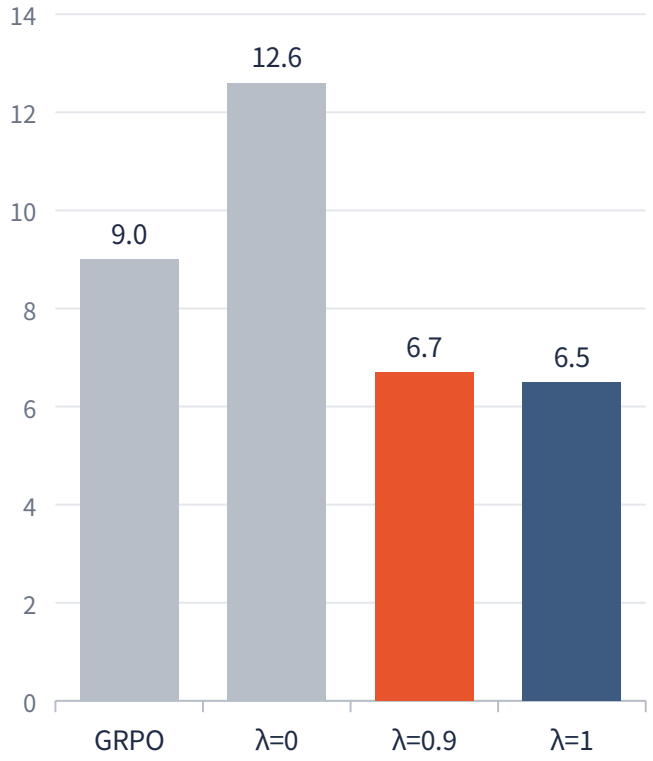
Critic success AUC

Within-group AUC 0.904, calibration error 0.006

Counterexamples: why not policy-side features?

- Action counts: pre-grasp AUC 0.773→0.862, but they act as a time proxy; negative credit leaks to perception
- LM hidden states: decode request index (R^2=0.80); 35% of |A| mass drifts to perception

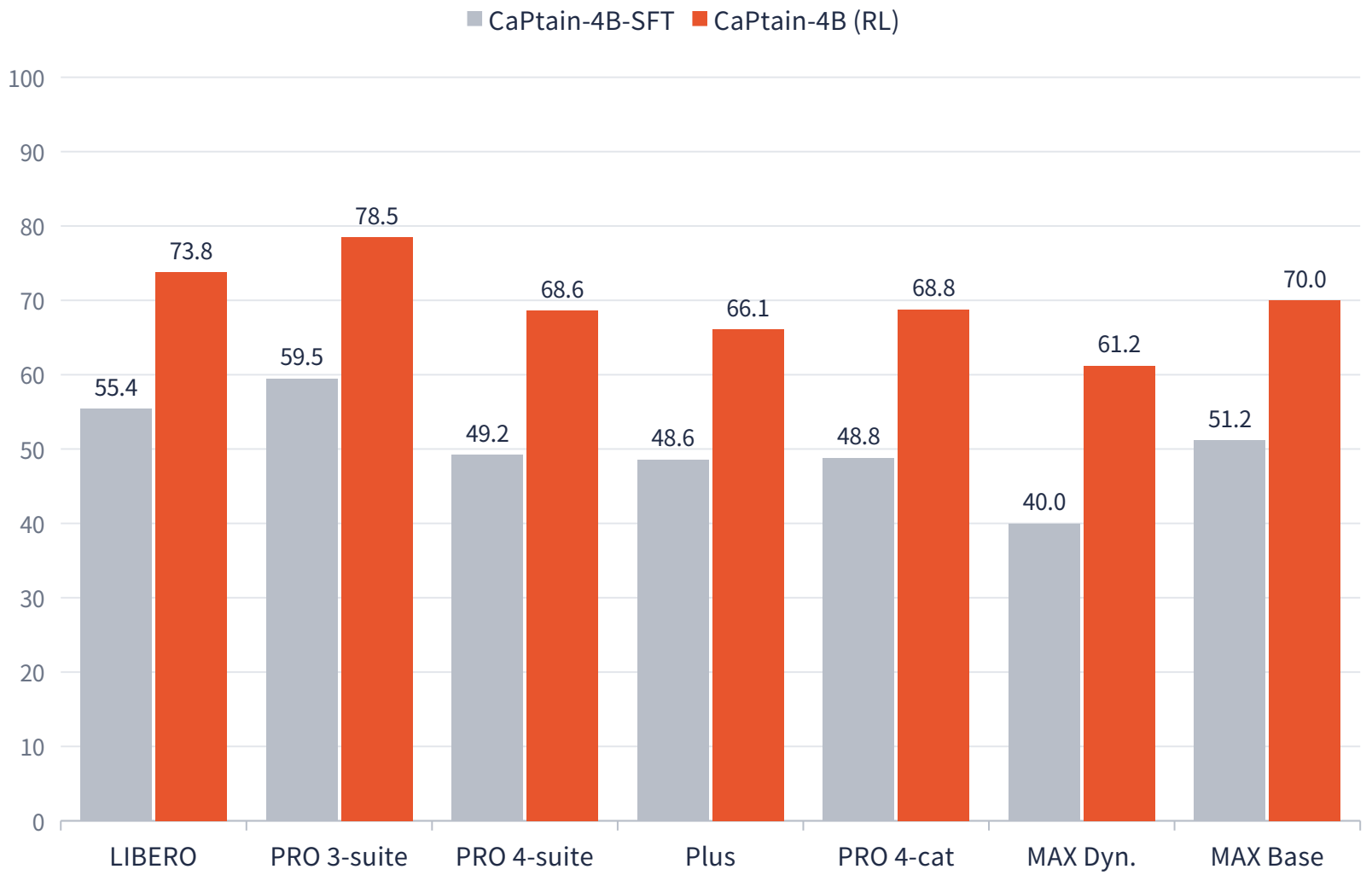
Gradient noise scale (k eps; lower better)



outcome-only GRPO 9.0k → simulator critic (λ=0.9) 6.7k; λ=0 is actually higher (12.6k)

Data: paper § 4.5, Appendix H Table 7 (offline 9B-policy rollouts: 15k episodes / 573k states)

RL consistently adds +17.6 to +21.2 points across the LIBERO family



+18.3

LIBERO host （55.4→73.8）

strict 52.1→67.5; +55 / −11 flips

7.1% → 2.1%

Fewer false finishes

Avg. requests 39.1 → 32.6

$p < 10^{-5}$

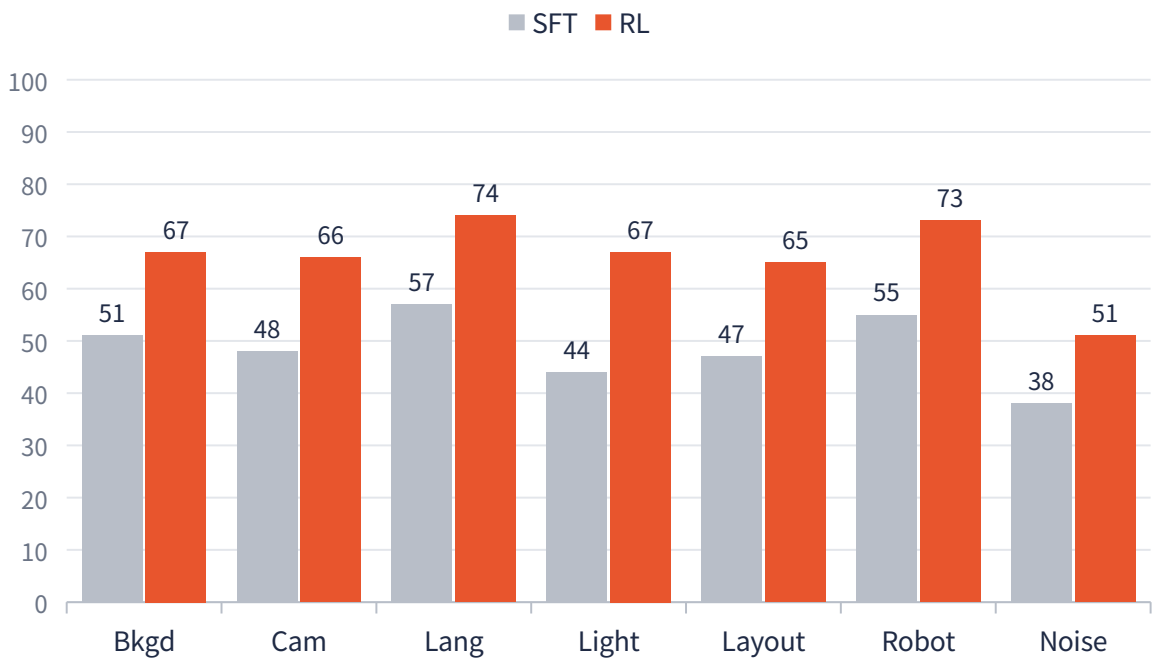
Paired McNemar, main suites

Base Qwen3.5-4B ⩽0.3% per suite

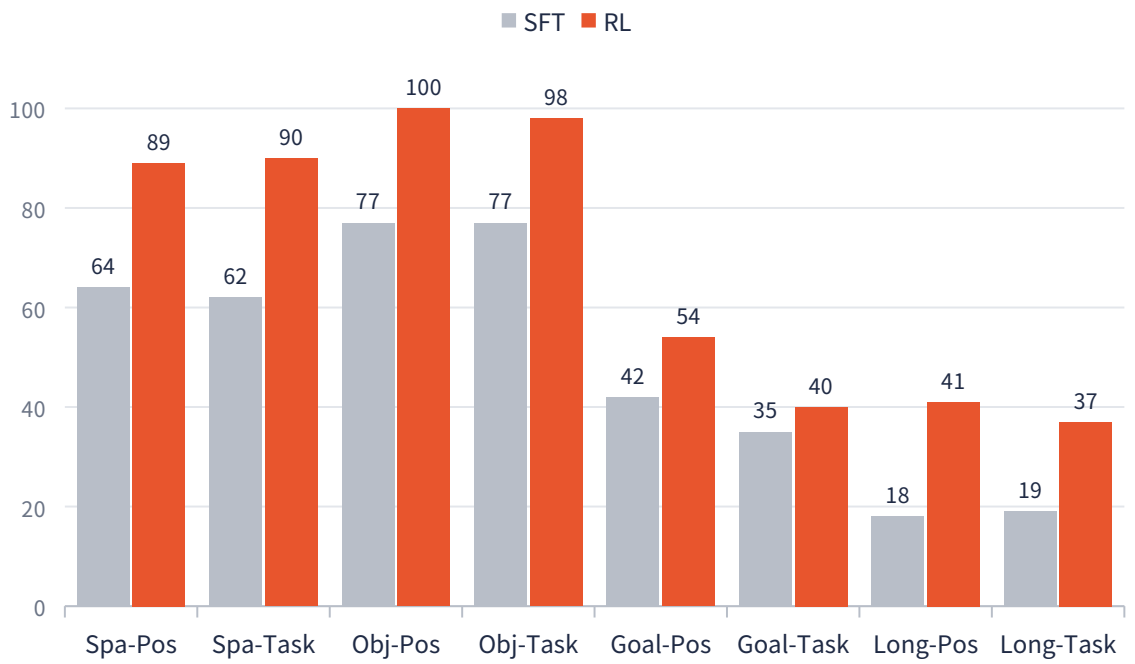
Data: paper Table 1 / Figure 2 (host success %); p-values in Appendix K Table 10

Gains hold under perturbation: significant on all seven LIBERO-Plus dimensions

LIBERO-Plus by perturbation dimension (host %)



LIBERO-PRO by suite × perturbation type (host %)



π0.5 — **97% → 12.8%**

Standard LIBERO → LIBERO-PRO position + task perturbations

RIPT-VLA — **68.4 vs 69.6**

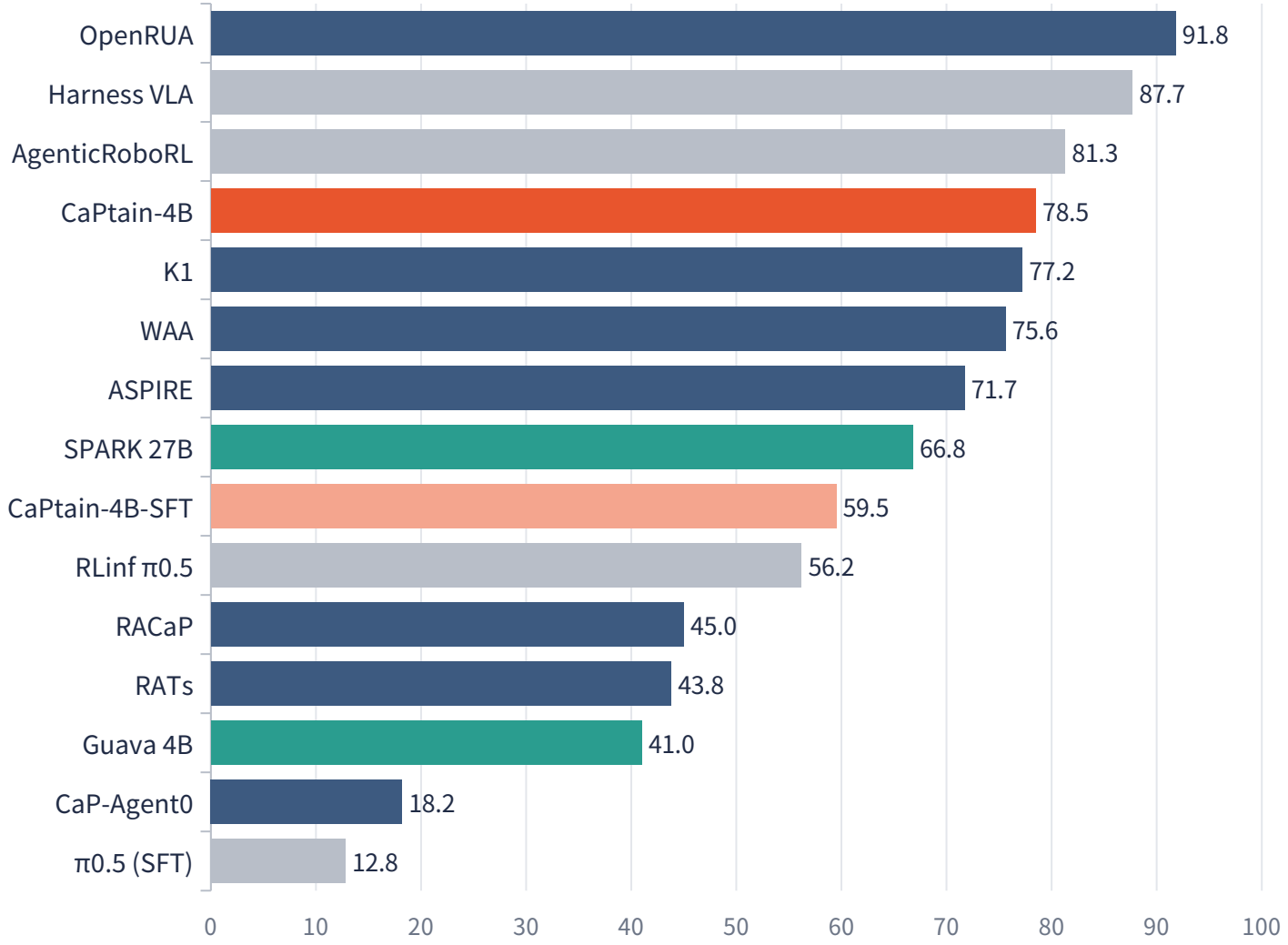
RL on VLAs: below base OpenVLA-OFT on LIBERO-Plus

CaPtain-4B — **90% retained**

LIBERO-Plus / LIBERO = 66.1 / 73.8; no LIBERO demos used

Data: paper Figure 3(b)(c), § 4.1, Appendix Q Table 14

A standalone 4B policy beats several frontier-model agents on LIBERO-PRO



- CaPtain (ours)
- Closed frontier-model CaP agents
- Open CaP agents (no VLA skills)
- End-to-end VLA / VLA-assisted agents

RL gain comparison (4-suite avg.)

+19.4

CaPtain • low-level primitives
49.2 → 68.6

+5.9

AgenticRoboRL • frozen VLA
65.1 → 71.0

Cost: 32.6 requests, 7.3k output tokens, ~52 GPU-seconds per episode ≈ **$0.03–0.06**

vs. SPARK / WAA / RACaP $0.18–0.24; OpenRUA $8–22

Note: different RL starting points; baseline protocols vary (episode budget, temperature, oracle, etc.)

Data: paper Table 2 (LIBERO-PRO, 3 suites); baselines from original papers; protocol diffs: App. K

Trained only on LIBERO, zero-shot: strong on single-arm tasks; bimanual and unfamiliar arms remain weak

83.3%

SimplerEnv WidowX

ManiAgent (GPT-5) 86.8 • PhysBrain 80.2 • π0.5 57.1

75.0

CaP-Bench

Gemini-3-Pro 73.5 • CaP-RL 7B 62.0; Lift/Stack 97.5% with no step limit

22.6%

RoboTwin 2.0 （Hard）

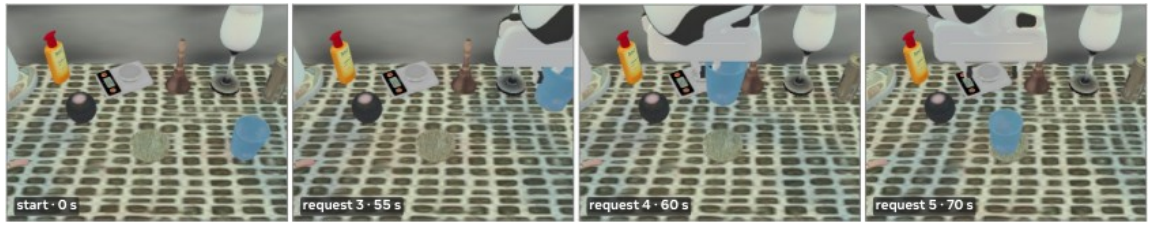
Single-arm subset 30.0%; in-domain π0.5 46.0 • OpenWAM-α 48.7

0.8%

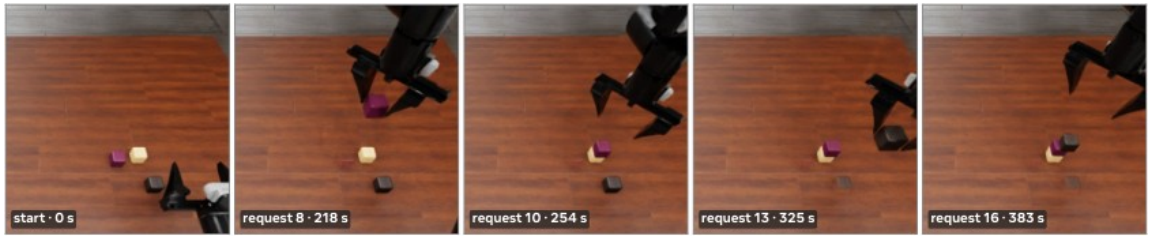
RoboDojo (dual ARX X5)

Score 1.7; GPT-6 Astra 29.0; in-domain VLA Score ⩽0.5

Failure attribution: RoboTwin tasks needing simultaneous bimanual motion 0/35; in RoboDojo, 21/46 first blocking failures are IK/reachability (X5 top-down grasp workspace only ~0.5 m); repeated verification also exhausts requests



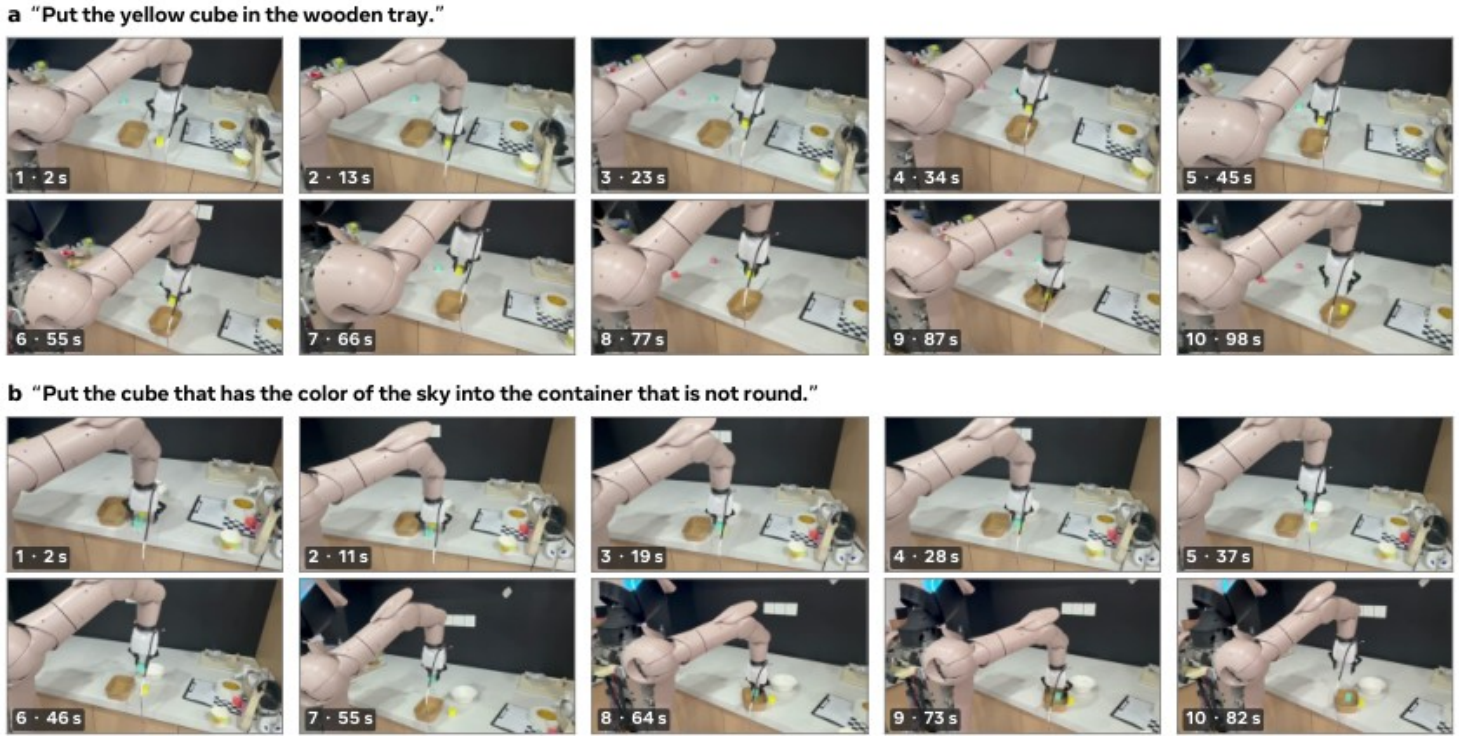
RoboTwin: rim grasp → measured-offset move → coaster (7 req, 77 s)

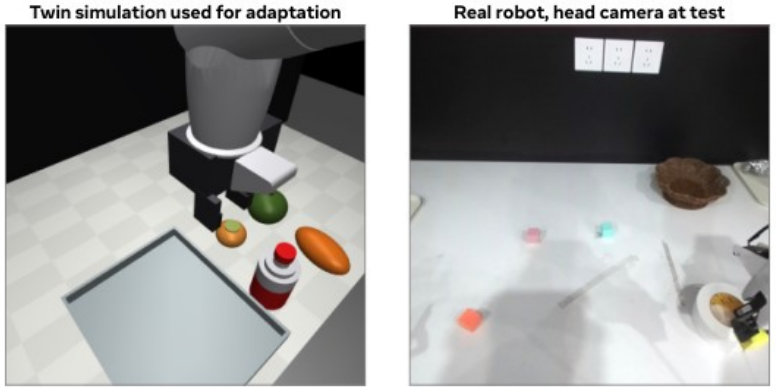


RoboDojo: force-stop recovery, 45° regrasp, 3-block stack (20 req, 413 s)

Data: paper Table 3, § 4.3, App. J; images: Figure 8 (RoboTwin), Figure 9 (RoboDojo)

Real robot: adapted only in a coarse digital-twin sim with zero real data; solves tasks with new objects and layouts





Left: XR1 twin sim (primitives, untextured) Right: real scene

Setup

- Astrabot XR1 + head RGB-D + wrist camera; same API / receipts, no critic

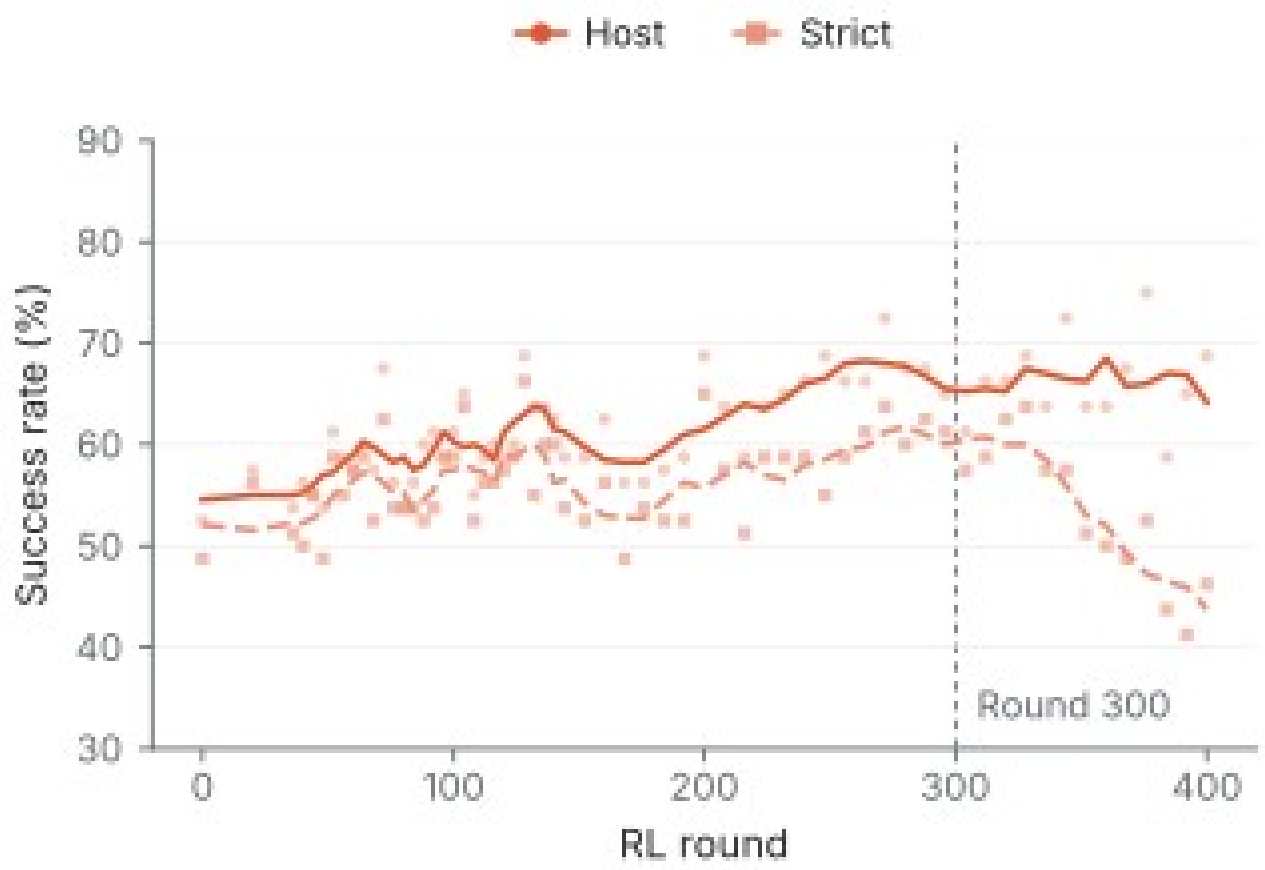
Observed behaviors

- (b) Reasons "sky color = light blue", "non-round = rectangular tray"
- Rejects unreliable masks, switches cameras, re-measures after placing before finish()

Only 2 qualitative success cases; no success-rate statistics

Images: paper Figure 4 (real episodes), Figure 5 (twin sim vs. real scene); trajectories in App. O

Caution: late in RL, “finish collapse” emerges: the policy over-verifies and refuses to stop



Symptom (after ~iteration 340)

host stays ≈65%; strict drops from ≈60% to **44%**

Lost episodes exhaust requests re-verifying already completed scenes

Mechanism: stopping gets no credit

- Success without finish still earns 0.25
- Post-success costs are capped (−0.45)
- Critic trained on host success gives V≈1 to all post-success states

Mitigation: the iteration-300 checkpoint is used as CaPtain-4B (with some cherry-picking);

the authors suggest a separate objective for stopping, e.g. success-and-finished

Image: paper Figure 7; analysis: Appendix I

Limitations: long horizons & bimanual, privileged sim state, action space, comparability

Long horizons & bimanual

libero_10 only +13.3 (p=0.13, not significant); simultaneous bimanual tasks 0/35; RoboDojo 0.8%

Relies on privileged sim state

With 2 cm pose noise, ΔV correlation falls to 0.49 (0.36 at 5 cm), and to only 0.21 with the policy's own perception → real-world online RL needs external state estimation

Constrained action space

Guarded Cartesian position primitives cannot apply sustained force (CaP-Bench Wipe infeasible); behavior coverage limited by SFT data

Training needs frontier models

SFT trajectories generated by GPT-6 Astra and others (654M tokens): not needed at test time, but essential for training

Evaluation comparability

Baselines from original papers, with differing episode budgets / temperatures / trial counts / oracle use; RoboDojo uses reachable layouts & custom adaptation

Speed & external services

LIBERO averages 417 s/episode (8 episodes sharing a GPU); perception relies on SAM 3 and Molmo2-8B, so “4B” is not the full compute

Takeaways: post-train manipulation like language agents

1

Low-level primitives are the source of RL gains

No skills or VLA: grasping, approach and placement live in code → RL optimizes manipulation itself (+19.4 vs. +5.9 for RL on a frozen VLA)

2

Asymmetric critic + same-time-index ΔV

Critic reads sim physics state, policy sees only cameras; elapsed time earns no credit, and perception gets only 1.3% of advantage mass

3

Engineering details make or break it

1,024-episode accumulation per step fights gradient noise; wide layouts prevent coordinate memorization; start states cover all tasks; stopping needs its own signal

4

Open problems

Termination (finish collapse), long horizons & bimanual, force-control tasks, and training's reliance on frontier-model data and privileged sim state

arXiv 2610.40799 • Code, SFT data and CaPtain-4B weights are open-sourced (per the paper)